Prof. Ashwani Mishra
Assistant Professor
CSE Department

1. Introduction to Node.js

What is Node.js?

Node.js is a JavaScript runtime environment that allows JavaScript to run on the server side.

Before Node.js:

- JavaScript only worked in the browser.

After Node.js:

- JavaScript can run on servers, APIs, databases, and applications.

Node.js is built on Chrome's V8 JavaScript Engine.

Example

Example: Online Shopping Website

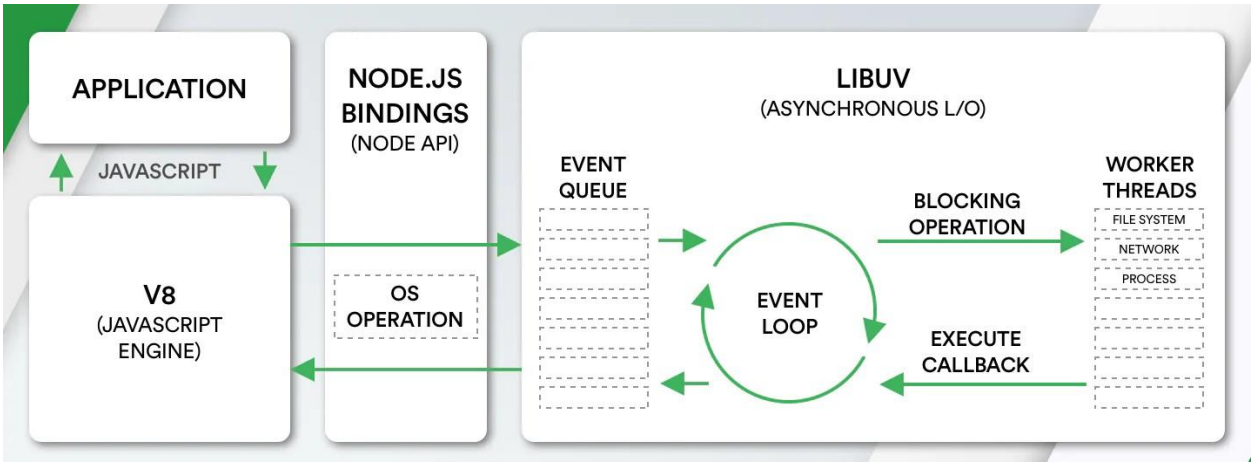
When user clicks Buy Now

Process:

- Browser sends request
- Server processes order
- Database stores order
- Server sends response

Node.js handles the server logic.

Node.js Architecture



Node.js uses

Event Driven Architecture

Components:

- Client Request
- Event Loop
- Non Blocking I/O
- Callback Functions

Flow:

Client Request

↓

Event Loop

↓

Process Task

↓

Return Response

Features of Node.js

Feature	Description
Asynchronous	Non-blocking execution
Fast	Built on V8 engine
Event Driven	Uses events and callbacks
Scalable	Handles many users

Simple Node.js Example

Create file

app.js

```
const http = require('http');


http.createServer((req, res)=>{
   res.write("Welcome to Node.js Server");
   res.end();
}).listen(3000);
console.log("Server running at http://localhost:3000");
```

Run:

node app.js

Output in browser

http://localhost:3000

Node.js Modules

Modules are **reusable blocks of code**.

Types:

1 Core modules
2 Local modules
3 Third-party modules

Core Module Example

```
const os = require("os");

console.log(os.platform());

console.log(os.arch());
```

2. Introduction to Express.js

What is Express.js?

Express.js is a web framework for Node.js.

It helps developers build:

- Web applications
- REST APIs
- Backend services

Express simplifies server creation.

Installing Express

Step 1 Create project

```
mkdir nodeproject

cd nodeproject
```

Step 2 Initialize project

```
npm init -y
```

Step 3 Install Express

```
npm install express
```

Simple Express Server

Create file

server.js

```
const express = require('express');


const app = express();
```

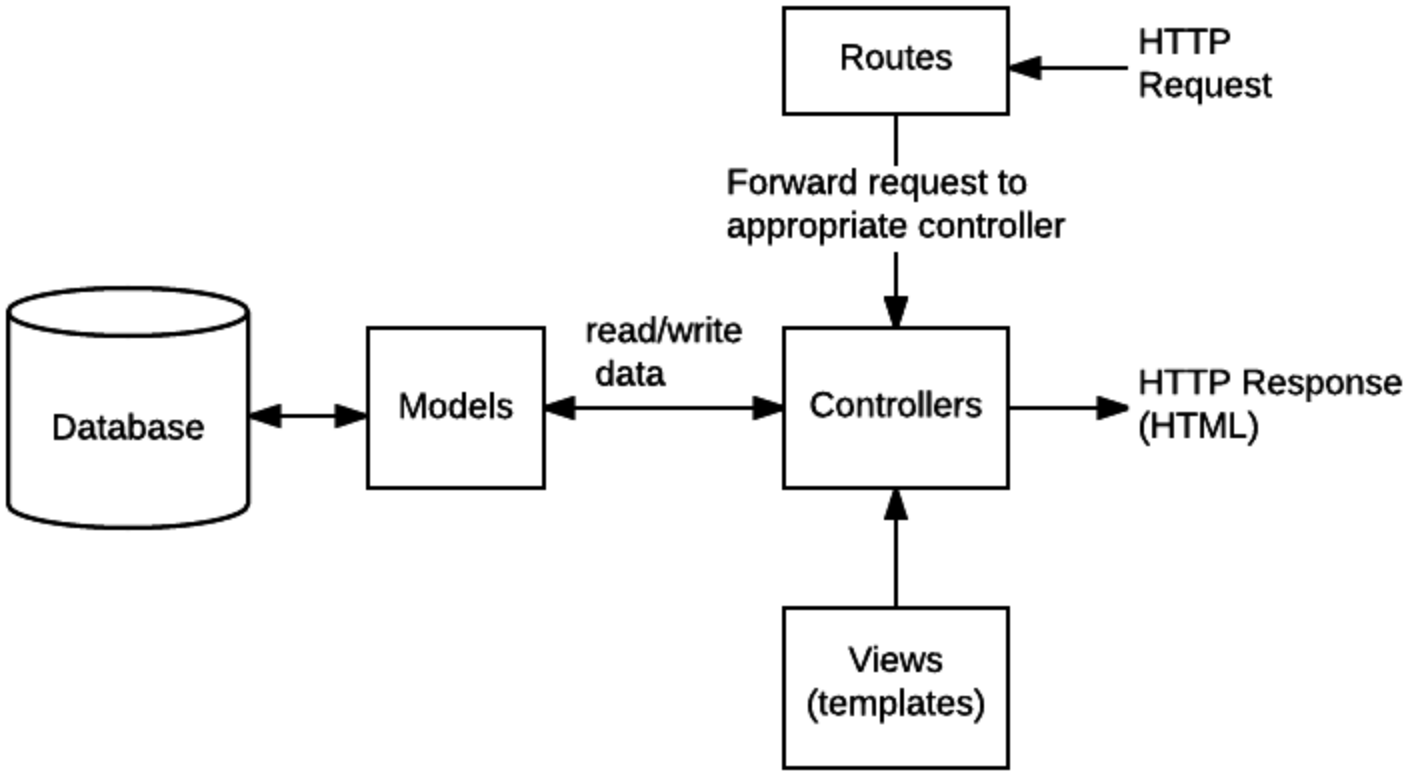
```
app.get('/', (req,res)=>{
  res.send("Hello from Express Server");
});

app.listen(3000, ()=>{
  console.log("Server running on port 3000");
});
```

Open browser:

http://localhost:3000

Express Application Architecture



Flow:

Client Request
↓
Middleware
↓
Route Handler
↓
Response

3. Middleware in Express

What is Middleware?

Middleware is a **function that executes between request and response**.

Uses:

- Logging
- Authentication
- Data validation
- Request processing

Middleware Flow

Request → Middleware → Route → Response

Example Middleware

```
const express = require("express");
const app = express();
app.use((req,res,next)=>{
  console.log("Middleware executed");
  next();
});
app.get("/",(req,res)=>{
  res.send("Home Page");
});
app.listen(3000);
```

<u>Output in terminal:</u>

Middleware executed

Types of Middleware:

Type	Use
Application middleware	Runs for all routes
Router middleware	Runs for specific routes
Error middleware	Handles errors

Third party middleware	Passport, Morgan

4. Routing in Express

What is Routing?

Routing means handling different URL requests.

Example routes:

/home

/about

/contact

Route Example

```
const express = require("express");
const app = express();
app.get("/",(req,res)=>{
 res.send("Home Page");
});
app.get("/about",(req,res)=>{
 res.send("About Page");
});
app.get("/contact",(req,res)=>{
 res.send("Contact Page");
});
app.listen(3000);
```

REST API Routing Example

Method	URL	Work
GET	/users	Get users
POST	/users	Add user
PUT	/users/:id	Update
DELETE	/users/:id	Delete

Example

```
app.post("/users",(req,res)=>{

  res.send("User Created");

});
```

5. Authentication with Passport.js

What is Authentication?

Authentication verifies **user identity**.

Example:

Login system.

User provides:

username

password

Server verifies credentials.

What is Passport.js?

Passport.js is an authentication middleware for Node.js.

It supports:

- Local login
- Google login
- Facebook login
- JWT authentication

Passport Authentication Flow:

User Login

↓

Passport Strategy

↓

Verify Credentials

↓

Login Success / Fail

Install Passport:

npm install passport passport-local express-session

Example:

```
const passport = require('passport');
const LocalStrategy = require('passport-local').Strategy;
passport.use(new LocalStrategy(
 function(username,password,done){
   if(username === "admin" && password === "123"){
      return done(null,{user:"admin"});
   }else{
      return done(null,false);
   }
 }));
```

6. Security in Express Applications

Common security risks:

- SQL Injection
- Cross Site Scripting
- Unauthorized access

Security methods:

1. Authentication
2. Authorization
3. Password hashing
4. HTTPS

Password Hashing Example

Install bcrypt

npm install bcrypt

Example:

```
const bcrypt = require("bcrypt");

const password = "12345";

bcrypt.hash(password,10,(err,hash)=>{

  console.log(hash);

});
```

7. Error Handling in Express

Error handling manages unexpected problems.

Example errors:

- Server crash
- Invalid data
- Database failure

Error Handling Middleware:

```
app.use((err,req,res,next)=>{
  console.error(err.stack);
  res.status(500).send("Something went wrong");
});
```

Example Error

```
app.get("/error",(req,res)=>{
  throw new Error("Test Error");
});
```

8. Logging in Node.js

Logging records application activities.

Example logs:

- User login
- Errors
- Requests

Morgan Logging **Middleware**

Install:

npm install morgan

Example:

```
const morgan = require("morgan");
app.use(morgan("dev"));
```

Example log:

```
GET /home 200 12ms
```